Test

Good, this is *exactly* the part that usually feels a bit hand-wavy, so let's slow it down.

I'll break it into two bits:

1. What that short-hand $\frac{\partial h}{\partial x} = \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial x}$ actually means.
2. What it means to "apply this from the output back to the input" in a neural network, and why that gives us the gradients we need.

---

1. The short-hand $\dfrac{\partial h}{\partial x} = \dfrac{\partial h}{\partial z} \cdot \dfrac{\partial z}{\partial x}$

We start with a composition of two functions:

- g takes x and produces z:
$$z = g(x)$$
- f takes z and produces $h(x)$:
$$h(x) = f(z) = f(g(x))$$

The chain rule says:
$$\frac{\partial h}{\partial x} = \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial x}.$$

What's going on?

- $\dfrac{\partial z}{\partial x}$: how much z changes when we nudge x a bit.
- $\dfrac{\partial h}{\partial z}$: how much h changes when we nudge z a bit.
- Multiply them to get the total effect of nudging x on h.

The "short-hand notation" bit is just: instead of always writing $g(x)$ and $f(g(x))$, we *rename* $z = g(x)$ and then write:

- h as a function of z
- and z as a function of x

So the derivative is written in a compact way as

$$\frac{\partial h}{\partial x} = \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial x}.$$

Nothing deep — just notation to make the chain rule look cleaner.

---

2. How this becomes back-propagation in a neural network

Now replace:

- x = the **input** of some layer
- z = the **output** (activation) of that layer
- h = the **loss** L in the very end (depends on the final network output)

A neural network is just a long composition of functions:

$$x; \to; \text{Layer 1}; \to; \text{Layer 2}; \to; \dots; \to; \hat{y}; \to; L.$$

To update the weights with gradient descent, we need derivatives like:

- how does the **loss** L change if we change a weight in layer 3?
- how does L change if we change a weight in layer 1?

Those weights affect L *indirectly* through all the later layers, so we can't differentiate in one shot. Instead we apply the same chain-rule idea:

back-propagation = apply

$$\frac{\partial h}{\partial x} = \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial x}$$

over and over, layer by layer, but starting from the loss and going backwards.

Concretely:

1. **Forward pass**
    - You compute all intermediate values: pre-activations, activations, prediction $\hat{y}$, loss L.
    - You *store* these values (because you will reuse them for derivatives).

2. **Backward pass (back-prop)**

   - Start at the end: compute $\dfrac{\partial L}{\partial \hat{y}}$ (how loss changes if prediction changes).
   - Then for the last layer weights $w^{(2)}$, use chain rule:

$$\frac{\partial L}{\partial w^{(2)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w^{(2)}}.$$

   - To go one layer earlier, say to hidden unit z_1, you apply chain rule again:

$$\frac{\partial L}{\partial w^{(1)} * 11} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w^{(1)} * 11}.$$

   - That's literally the chain rule in the short-hand form:

$$\frac{\partial L}{\partial w^{(1)} * 11} = \underbrace{\frac{\partial L}{\partial \hat{y}}}_{} * \text{error at output} \cdot \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{} * \text{weight to next layer} \cdot \underbrace{\frac{\partial z_1}{\partial w^{(1)} * 11}}_{\text{local derivative at this neuron}}.$$

So when the text says:

> "We first differentiate the loss with respect to the final output $f_\theta(x)$, then propagate this derivative backwards through the last layer, then through the previous layer, and so on, each time multiplying by the relevant local derivative…"

it means exactly:

- compute the derivative at the **end** once (loss vs. final output),
- then repeatedly apply the chain rule to push that derivative back through each layer,
- at every step you just multiply by simple local derivatives that you know (weights, activation derivatives, etc.).

Why is this *efficient*?

Without back-prop, if you tried to differentiate L with respect to each weight separately, you would recompute a lot of the same stuff again and again.

Back-prop reuses:

- the **forward activations** (from the forward pass),
- the **backward error terms** (the deltas) as you move backwards.

So instead of "compute a huge derivative from scratch for each weight", you:

1. Do **one** forward pass.
2. Do **one** backward pass, and in that single backward sweep you get all $\frac{\partial L}{\partial w}$ for all weights at once.

That's what the sentence "This is what makes back-propagation an efficient method for obtaining the gradients needed in gradient descent" is saying: back-prop is just the chain rule organised cleverly so we don't waste work.

If you like, I can draw a tiny toy network and we can compute one or two derivatives by hand using exactly this chain rule idea so you see the mechanics.